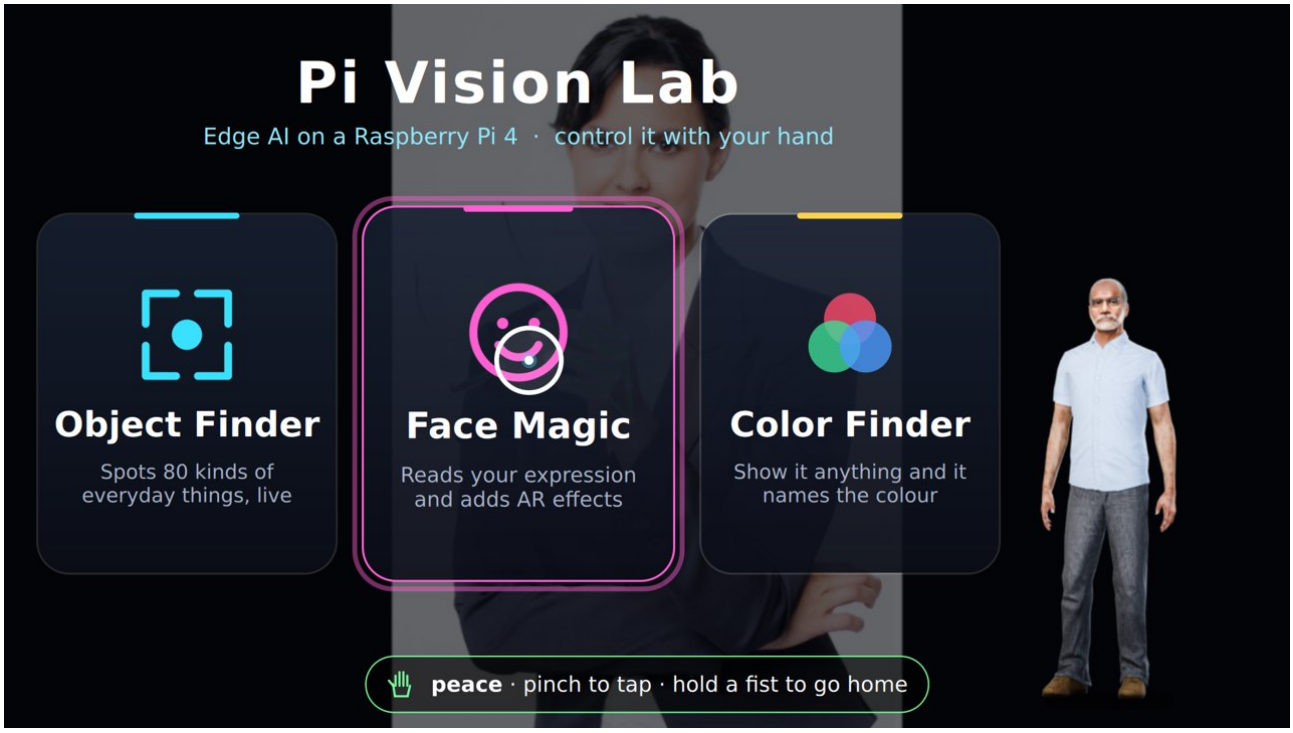
Pi Vision Lab

How it works: the technology behind an offline, gesture-controlled AI kiosk on a Raspberry Pi 4



The home screen: three AI demos, a hand-controlled cursor and a 3D presenter.

This document explains the ideas behind the system for a technical reader: what the camera sees, which AI models run, how detection and tracking work, how a hand becomes a mouse, and how the 3D presenter is made. It doesn't go into the program code or the user-interface framework. Source code and setup instructions are at **github.com/hareendranmg/pi-vision-lab**.

Version 1.0 · October 2026

1. What it is

Pi Vision Lab is a self-contained demo station. A camera watches the person in front of a screen, and AI models running **on the Raspberry Pi itself** understand what they see, about ten times a second. There is no internet connection, no cloud service and no special AI chip. Visitors don't need to touch anything: they control the screen by pointing and pinching in the air.

Screen	What the visitor does	What the AI does
Object Finder	Holds up everyday things	Finds and labels 80 kinds of objects with boxes
Face Magic	Looks at the camera, picks an effect	Maps 478 points on the face, reads the expression, adds realistic effects
Color Finder	Points a finger at something	Names the colour just past the fingertip
Everywhere	Raises a hand	Tracks 21 points on the hand to drive an on-screen cursor

A **3D presenter** (an animated person rendered from a real photo) walks onto the screen, comments on what is happening in speech bubbles, and walks off when it's quiet. The station has no speaker or microphone; everything is visual.

1.1 Hardware

Part	Model	Role
Computer	Raspberry Pi 4, 8 GB	Runs everything: camera capture, all AI models, the display
Camera	Raspberry Pi Camera Module 3 (wide)	960×540 colour video, continuous autofocus
Display	Pi touch display or any HDMI monitor	Full-screen app; touch also works
Software	Raspberry Pi OS (Debian 13), Python	Starts automatically when the Pi powers on

> **Privacy by design:** camera images are processed in memory and discarded. Nothing is recorded, stored or sent anywhere.

2. The big picture

The whole system is one loop that repeats for every camera frame:

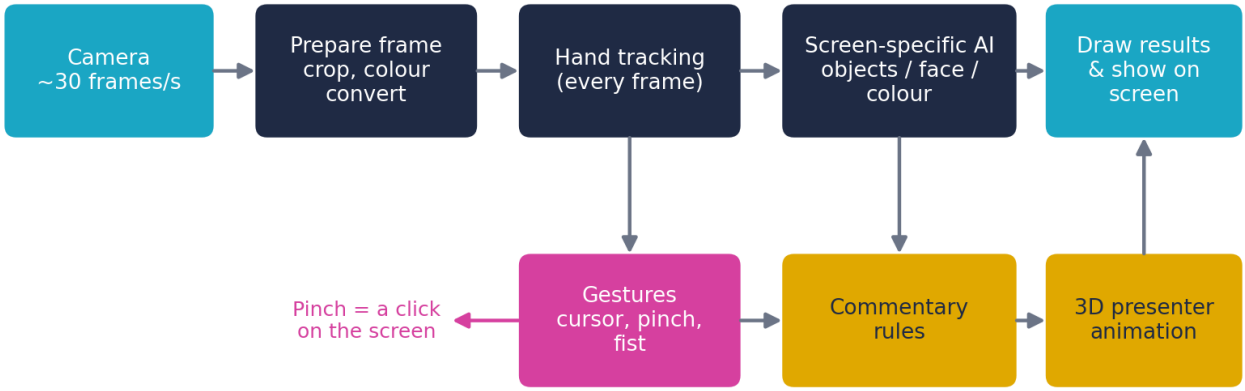


Figure 1: one pass of the main loop. Hand tracking always runs; the other model depends on the screen.

- **Capture.** The camera delivers a new picture about 30 times a second. The loop takes the newest one.
- **Prepare.** The picture is trimmed to the shape of the screen and converted to the colour format the models expect.
- **Hand tracking** runs on every frame on every screen, because the hand is how the visitor controls the system.
- **Screen-specific AI** runs next: object detection, face analysis or colour naming.
- **Display.** Results are drawn onto the picture, which is then mirrored (so the screen acts like a mirror) and shown.
- **In parallel,** the gesture logic turns hand movement into clicks, and the commentary rules decide whether the presenter should say something.

One trip round the loop takes about 85 to 120 milliseconds on the Pi 4, which gives 8 to 12 updates a second depending on the screen.

3. A short primer on the AI

A **model** here is a trained neural network: a large, fixed set of numbers learned from many example images. Running a model on a new picture (called **inference**) is a long series of multiplications and additions that turns pixels into useful outputs. Training was done in advance by Google; the Pi only runs the finished models.

The system uses three kinds of vision task:

Task	Question it answers	Output
Detection	Is there an object here, and where?	Boxes with a label and a confidence score
Landmark estimation	Exactly where are the key points of this hand or face?	A list of (x, y) points: 21 for a hand, 478 for a face
Classification by rules	What does this measurement mean?	Names such as "pinch", "happy" or "Teal", from simple maths on the points or pixels

3.1 The models

Model	Made by	Input	Output	Time on Pi 4
BlazePalm (palm detector)	Google MediaPipe	192×192 image	Palm box + 7 key points	~45 ms
Hand landmark model	Google MediaPipe	224×224 hand crop	21 hand points + "is it a hand?" score	~39 ms
BlazeFace (short range)	Google MediaPipe	128×128 image	Face box + 6 key points	~5 ms
Face mesh model	Google MediaPipe	256×256 face crop	478 face points	~32 ms
EfficientDet-Lite0	Google (TensorFlow)	320×320 image	Up to 25 boxes, 80 COCO classes	~70 ms

All models run with **LiteRT** (formerly TensorFlow Lite), Google's runtime for small devices, using **XNNPACK**, a library that uses the Pi's NEON vector instructions to do many multiplications at once. The object detector is **quantised** to 8-bit integers, which makes it smaller and faster with little loss of accuracy.

> **Why not MediaPipe directly?** Google's MediaPipe library would normally run these models. Its current builds require a processor feature (ARMv8 AES instructions) that the Pi 4's chip doesn't have, so it crashes on start-up. The project re-creates the same processing steps around the same model files, using LiteRT, which runs on any 64-bit ARM chip.

4. Detect once, then track

Hands and faces are found in **two stages**. This is the key idea that makes real-time tracking possible on a small computer.

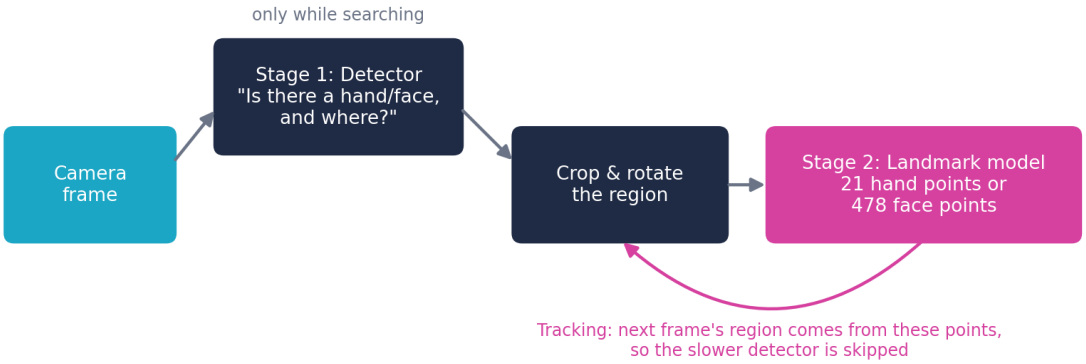


Figure 2: the two-stage pipeline used for both hands and faces.

- **Stage 1, the detector,** looks at the whole (shrunk) picture and answers "is there a hand or face, and roughly where?". It also gives a few key points, such as the wrist and middle knuckle, or the two eyes.
- Those points tell us the size and the angle of the hand or face. The system cuts out a **square region around it, rotated upright**, so the next model always sees the hand or face the right way up and at a similar size.
- **Stage 2, the landmark model,** looks only at that cut-out and places the detailed points precisely. It also returns a score saying whether a hand or face is really still there.
- **Tracking:** on the next frame the region is taken from the points just found, because people don't move far in a tenth of a second. The slower detector only runs again if the landmark model loses the target.

Because the cut-out is small and upright, the landmark model can be very accurate, and skipping the detector saves 45 ms per frame while a hand is being tracked.

4.1 Finding small faces

The face detector is designed for faces close to the camera and sees the picture at only 128×128 pixels. A visitor two metres away would cover just 12 pixels: too small to find. So while searching, each frame checks a different region: the whole picture, or one of six zoomed-in tiles. Each check costs only about 5 ms, and a face anywhere in view is found within about a second.

Face search: the whole frame (gold) + 6 zoomed tiles, one checked per frame

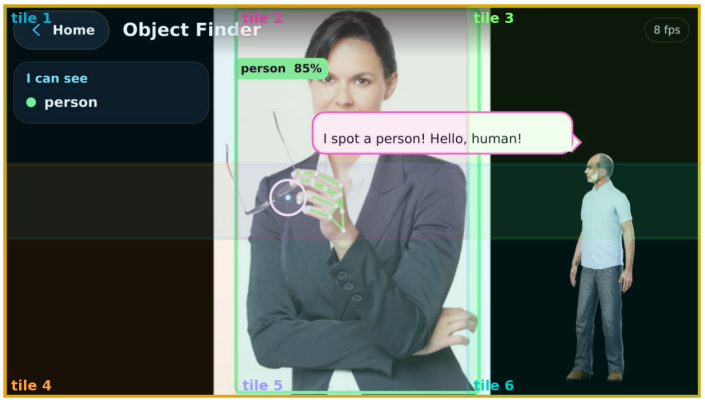


Figure 3: the face search regions. Once a face is found, tracking takes over and the search stops.

5. Hands become a mouse

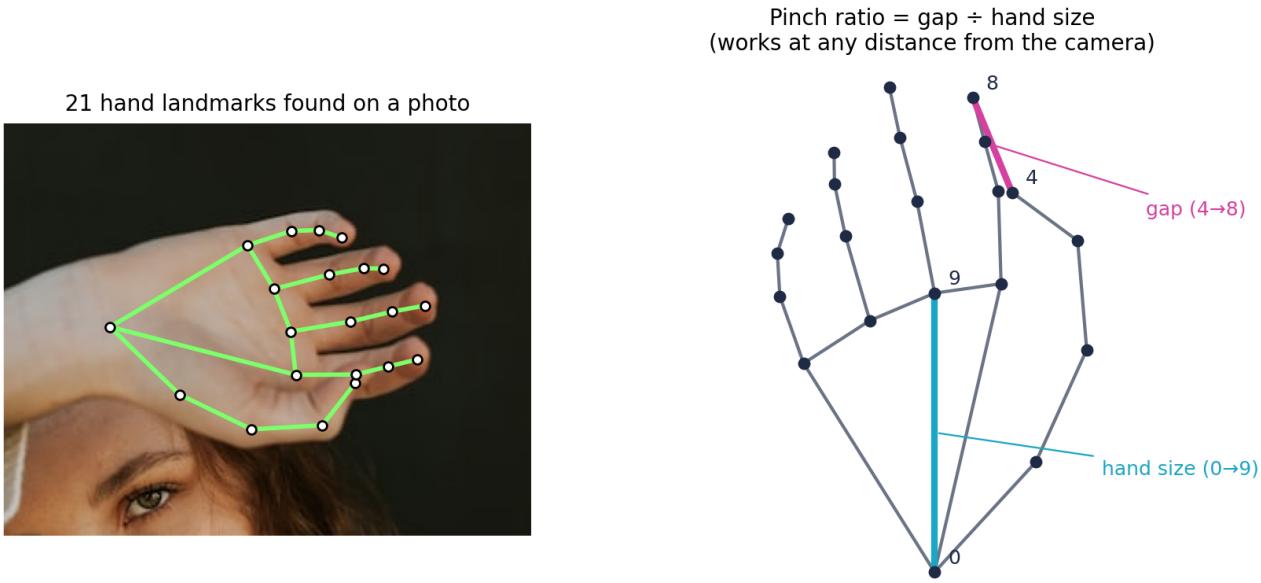


Figure 4: the 21 hand landmarks (left) and the measurements used for pinching (right).

5.1 The cursor

The cursor sits at the point midway between the thumb tip and the index fingertip. This point hardly moves when the fingers pinch together, so clicking doesn't knock the cursor sideways. The middle part of the camera view is mapped to the whole screen, so the visitor doesn't have to reach to the edges. A **One Euro filter** smooths the position: strongly when the hand is still (no jitter), lightly when it moves fast (no lag).

5.2 The pinch (click)

The system measures the **pinch ratio**: the gap between thumb tip and index tip divided by the size of the hand. Dividing by hand size means the same pinch works whether the visitor stands close to the camera or far away.

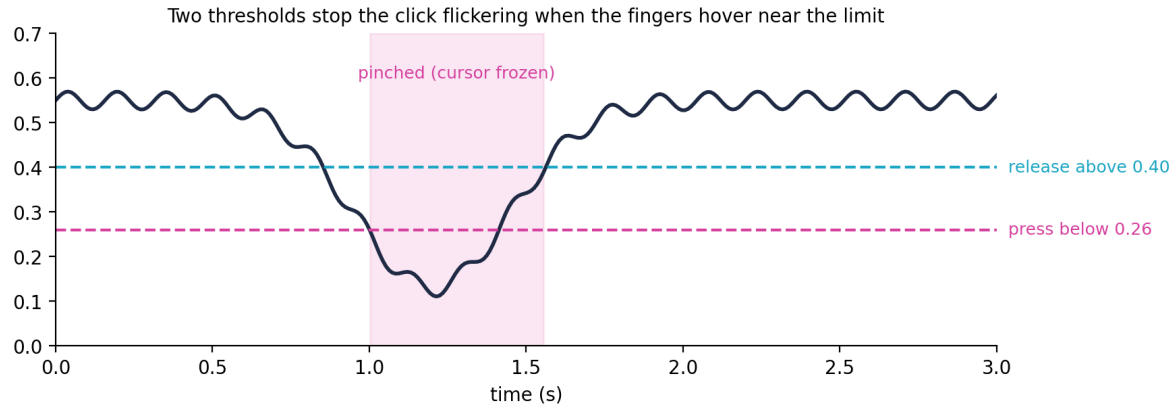


Figure 5: hysteresis. A click starts below 0.26 and ends only above 0.40, so small wobbles near one limit don't cause repeated clicks.

- **Two thresholds (hysteresis):** press below 0.26, release above 0.40.
- **Debounce:** a press or release must be seen on two frames in a row, so one misread frame can't click.

- **Aim freeze:** while pinched, the cursor stays where it was about 0.12 s before the fingers closed, which is where the visitor was aiming.
- **No false pinch from a resting thumb:** if the thumb tip is nearer the index finger's middle joint than its tip, it isn't counted.
- **Visual feedback:** the cursor fills up as the fingers approach, and ripples when the click happens.

The click is passed to the screen as an ordinary mouse event, exactly like a real mouse or a touch, so every button works the same way for hands and fingers.

5.3 The fist (go home)

A fist is recognised when at least four fingertips are folded in close to the centre of the palm. Holding it for one second returns to the home screen. An orange ring counts down, and short misreads (under 0.35 s) don't restart the timer. Other hand shapes (thumbs up, peace sign, rock sign, pointing, open palm) are recognised in the same way, by comparing which fingers are extended, and the presenter reacts to them.

6. Reading faces

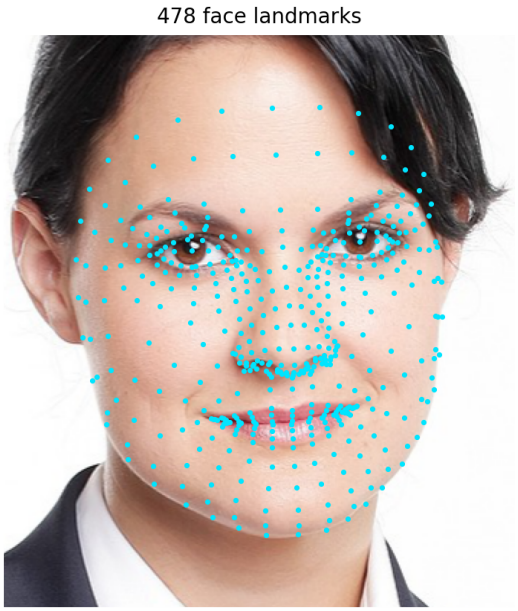


Figure 6: the 478-point face mesh on a stock photo.

The face mesh gives the position of 478 points covering the eyebrows, eyelids, lips, nose, jaw and the irises. Expressions are not read by another AI model; they are measured from the geometry of these points. To ignore head tilt, the points are first rotated so the eyes are level, then measured relative to the distance between the eyes.

Expression	What is measured
Smile	Mouth width plus how far the mouth corners rise above the centre of the lips, compared with this person's own resting face
Surprise	Eyebrows raised high above the eyes, plus an open mouth
Mouth open / talking	Gap between the upper and lower lip
Eyes closed / sleepy	Eye opening height divided by eye width (the "eye aspect ratio")
Wink	One eye clearly more closed than the other

The smile measure adapts to each person: it quickly learns a visitor's resting mouth shape but adapts only slowly upwards, so someone who walks up already smiling is still recognised as happy.

6.1 Face Magic effects



Figure 7: all six effects on two stock photos: mesh, masquerade, crown, shades, glam and ice eyes.

Two techniques make the effects look real:

- **Real photographs, fitted to the face.** The masquerade mask and the crown are photographs of real objects with transparent backgrounds. Each has anchor points (the mask's two eye holes, the crown's headband). Every frame, the photo is scaled, rotated and moved so its anchors line up with the eyes or the forehead.
- **Recolouring the visitor's own pixels.** Glam make-up, ice-blue eyes and sunglasses are painted using the mesh outlines (lips, eyelids, cheeks, irises). The new colour is multiplied by the brightness already there, so the visitor's real skin texture, shadows and lighting remain visible. Eyelids are taken into account, so a half-closed eye shows a half-covered iris.

Points are smoothed between frames so the props don't shimmer, while still following quick head movements.

7. Finding objects

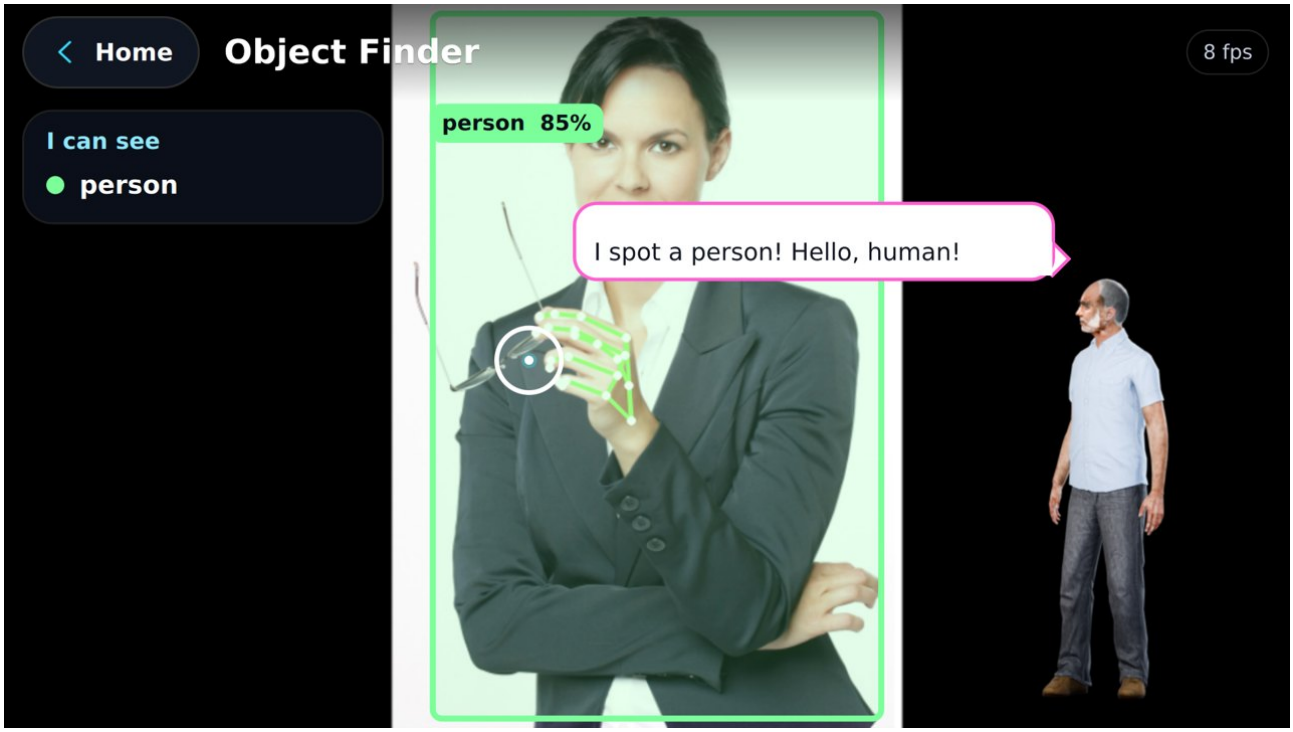


Figure 8: Object Finder showing a detected person and the running list.

Object Finder uses **EfficientDet-Lite0**, a compact detector trained on the **COCO** dataset, which covers 80 everyday categories: people, cups, bottles, phones, laptops, books, chairs, animals, food, vehicles and more. The picture is resized to 320×320 pixels and the model scores thousands of possible boxes at once.

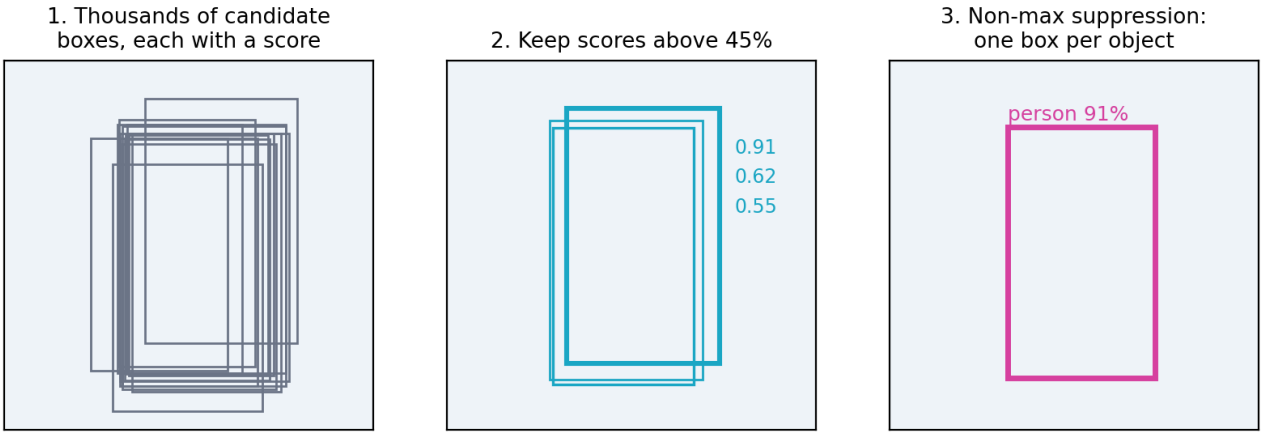


Figure 9: from thousands of candidates to one box per object.

- Boxes scoring below **45% confidence** are dropped.
- **Non-maximum suppression (NMS)** removes boxes that overlap a better-scoring box of the same object, leaving one box each.
- The detector runs on every **second** frame; in between, the last boxes are reused. This keeps hand tracking fast and smooth, and the boxes are still refreshed about four times a second.

8. Naming colours

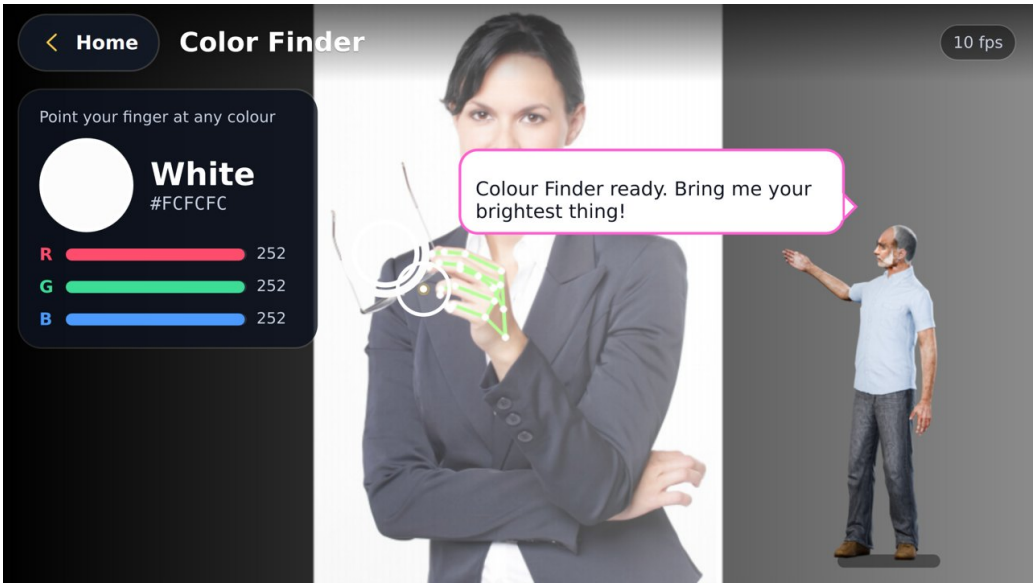


Figure 10: Color Finder. The circle follows the finger and the panel shows the colour.

- **Where to look:** the system takes the direction of the index finger (from its middle joint to its tip) and samples a point a little beyond the fingertip, so it measures the object being pointed at, not the finger. With no hand in view it uses the centre of the screen.
- **What colour it is:** it takes the **median** colour of a small patch at that point. The median ignores odd pixels such as shiny reflections.
- **What to call it:** the colour is converted to **CIELAB**, a colour space designed so that equal distances look equally different to the human eye. It's then compared with 33 named colours, and the nearest one wins.

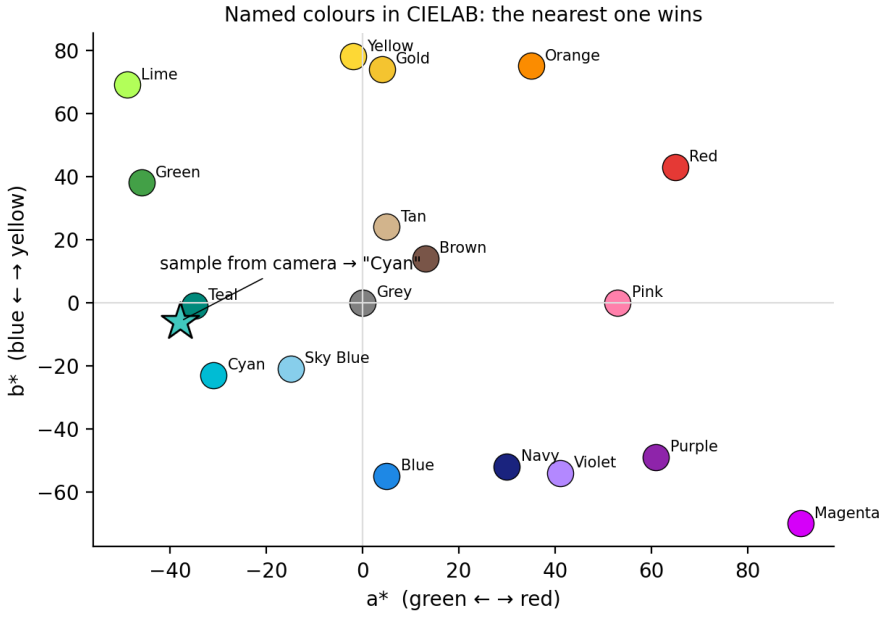


Figure 11: named colours placed in CIELAB (the lightness axis is not shown). The camera sample takes the nearest name.

> Camera colour depends on the lighting: a white wall under warm lamps can be named "Beige". Even, neutral light gives the most accurate names.

9. The 3D presenter

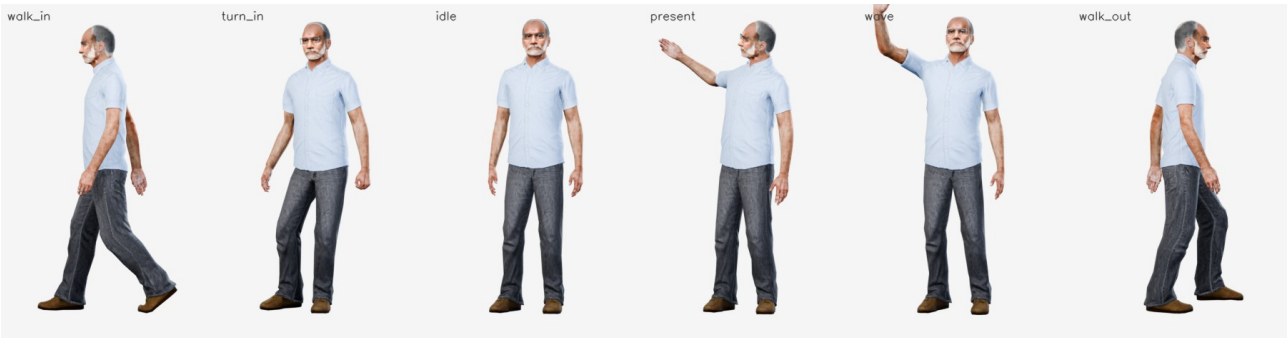


Figure 12: frames from six of the presenter's ten clips: walking in, turning, standing, presenting, waving and walking out.

A photorealistic, animated 3D person is far too heavy to render live on the Pi's small graphics chip. So the presenter is **pre-rendered**: every movement was rendered once on a PC with a graphics card, saved as transparent images, and the Pi simply plays them back like a flip-book. This costs almost nothing while the AI is running.

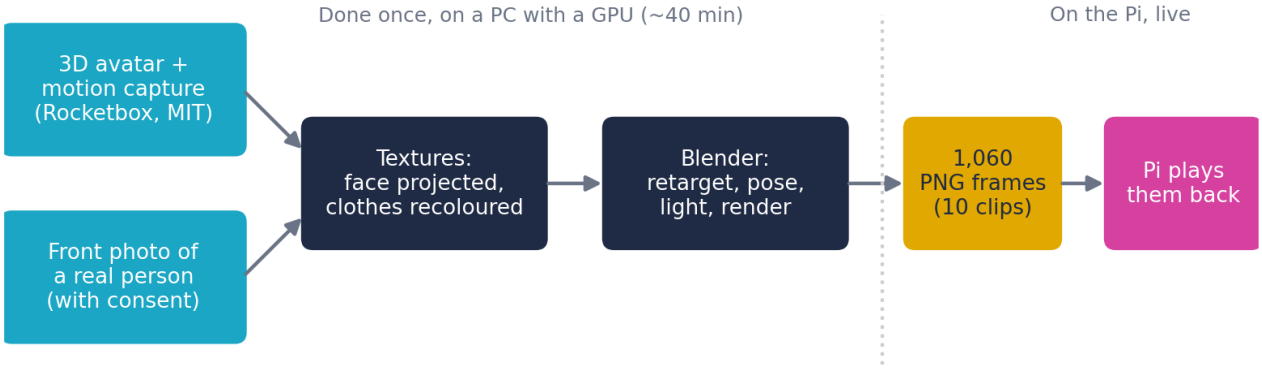


Figure 13: how the presenter is produced and played.

9.1 How the character was built

- **Body and movement:** the 3D model, skeleton and motion-capture animations come from **Microsoft Rocketbox**, a free (MIT-licensed) library of realistic people. Two models were combined: one body wearing a half-sleeve shirt and trousers, and one bald head.
- **Likeness:** a front photo of a real person (used with his permission) was mapped onto the head's texture. The same 478-point face mesh used in Face Magic finds matching points on the photo and on the flattened head texture. The photo is then cut into small triangles and each one is warped into place, and the edges are blended into the surrounding skin.
- **Clothes and details:** the shirt and trousers were recoloured while keeping their folds, the skin tone and eye colour were matched to the photo, and rectangular glasses were modelled.
- **Animation transfer (retargeting):** the motion-capture data was recorded on a skeleton standing in a T-pose, but the model rests in an A-pose. Each bone of the model is told to copy the real-world angle of the matching bone in the recording, which makes the difference cancel out.
- **Rendering:** Blender, a free 3D program, rendered ten clips (walk in, turn, idle, two talking styles, excited, presenting, wave, turn away, walk out) at 30 frames per second with studio lighting: 1,060 images in total. Standing clips loop seamlessly, because the software picks

the stretch of motion whose last pose best matches its first.

9.2 Playback and behaviour

- When there is something to say, he walks in from the right, turns to face the audience (angled slightly towards the content), and talks with gestures while a speech bubble types out the text.
- While walking, the picture slides across the screen at the same speed as the stride in the animation, so the feet don't appear to skate.
- Clips blend into each other over a fifth of a second, so changes of movement look smooth.
- After about 25 seconds with nothing new, he turns and walks off; he returns when something happens.

9.3 What he says

The commentary is **rule-based**, with no language model: events from the vision system trigger short lines picked at random from prepared lists. Rules keep it natural: at least 5 seconds between lines, a cool-down for each topic, and no repeats of the last eight lines.

Event	Example
A demo screen is opened	"Face Magic! I track 478 points on your face, live."
An effect is chosen	"Your Majesty! That crown really suits you."
An expression is held	"Love that smile!" / "Did you just wink at me?"
A new object is seen for over a second	"I spot a cup! Coffee or tea? I run on electricity myself."
A colour is held steady	"That's Turquoise, like tropical water!"
A hand sign	"Thumbs up right back at you!"
Someone arrives and raises a hand	"Oh hi there! Pinch to click, make a fist to go home."

10. Making it fast enough

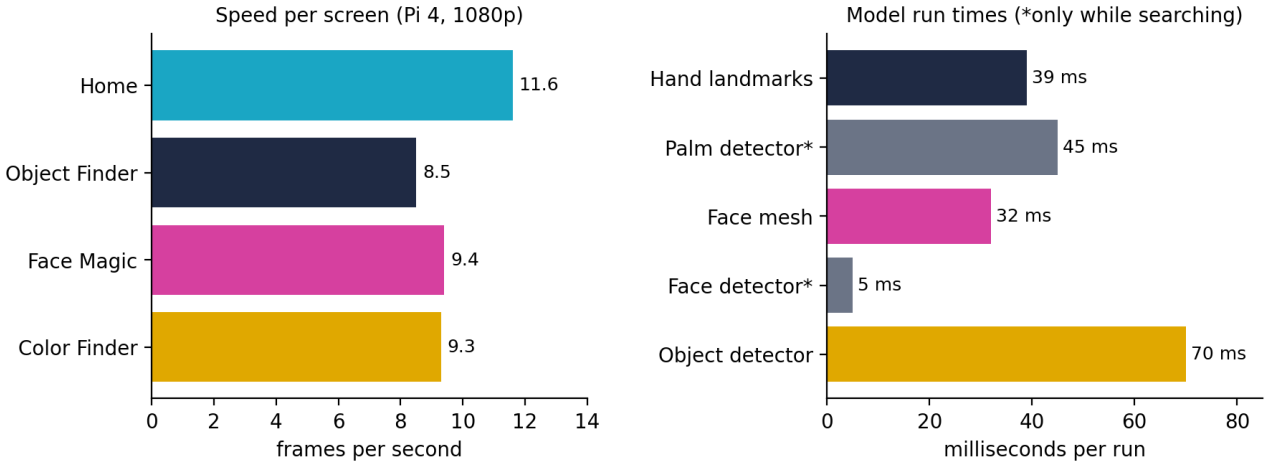


Figure 14: measured speed on a Raspberry Pi 4 driving a 1080p monitor, with the presenter on screen.

The Pi 4 has four processor cores and no AI accelerator, so every millisecond counts. The main techniques:

Technique	Effect
Detect once, then track (section 4)	The slow detectors run only while searching; tracking costs one landmark pass
Small, upright crops	Landmark models work on 224×224 or 256×256 pixels instead of the whole frame
Object detection on alternate frames	Halves the heaviest model's cost; boxes still update ~4 times a second
One face-search tile per frame	Finds distant faces for ~5 ms per frame instead of ~35 ms
Three worker threads per model	Stops two models fighting over four cores and starving the display
Graphics chip does the scaling	The camera picture and presenter are enlarged to full screen by the GPU, not the processor
Presenter frames loaded at start-up	~0.7 GB of images decoded once in the background, never during the show
8-bit quantised object model	Smaller and faster than the full-precision version

Together these took the Object Finder from about 5 to 8.5 updates a second, and Face Magic from about 6 to 9.4. On the smaller 800×480 touch display it's faster still, because there are fewer pixels to draw.

11. Running as a kiosk

- **Starts by itself:** the Pi logs into its desktop automatically and launches the app full-screen about 20 seconds after power-on. Screen blanking is turned off.
- **Restarts by itself:** a small watchdog script relaunches the app within 3 seconds if it ever stops.
- **Works offline:** all models and images are stored on the Pi's memory card. No network, account or key is needed.
- **Easy to reproduce:** one setup script installs everything on a fresh Raspberry Pi: system packages, the Python environment, the AI models and the kiosk settings. The presenter's rendered clips are included, so the 3D software is only needed to change him.
- **Remote viewing:** the screen can be watched and controlled over VNC from a PC on the same network.

12. Glossary

Term	Meaning
Anchor points	Known points on a prop photo (such as eye holes) that are lined up with points on the face
CIELAB	A colour space in which distance matches how different two colours look to people
COCO	A widely used dataset of everyday photos labelled with 80 object categories
Confidence score	The model's estimate (0–100%) that a detection is correct
Debounce	Requiring a change to be seen on several frames before acting on it
Detector	A model that finds where objects are and draws boxes around them
Hysteresis	Using different thresholds to switch on and off, to avoid flickering
Inference	Running a trained model on new input
Landmarks	Precise key points on a hand or face, such as fingertips or lip corners
LiteRT / TensorFlow Lite	Google's runtime for running AI models on small devices
Motion capture	Movement recorded from a real actor and stored as bone angles over time
Non-maximum suppression	Removing overlapping boxes so each object is reported once
One Euro filter	A smoothing filter that adapts to speed: steady when still, quick when moving
Quantisation	Storing a model's numbers as 8-bit integers to make it smaller and faster
Retargeting	Transferring an animation from one skeleton to another
XNNPACK	A library that speeds up neural-network maths on ARM and x86 processors

Credits: AI models by Google (Apache 2.0). 3D avatar and animations from Microsoft Rocketbox (MIT). Crown photo by Gryffindor (CC BY-SA 3.0) and mask photo by Alexas_Fotos (CC0), via Wikimedia Commons. The presenter's likeness is used with permission. Stock sample photos from Google MediaPipe.